



Objektorienterad Pascal-parser i Java

Edinson Uribe

18 juli 2026

DVGC01 Programspråk

Datavetenskap

Fakulteten för hälsa-, teknik- och naturvetenskap

Innehåll

1	Inledning	2
2	Hur parsern används	2
3	Objektorienterade tekniker och designmönster	3
3.1	Inkapsling istället för globalt tillstånd	3
3.2	Konstruktörsinjektion	3
3.3	Värdeobjekt	3
3.4	Enum med data istället för separata uppslagstabeller	3
3.5	Separation av begrepp	3
3.6	Inkapslade samlingar istället för parallella arrayer	4
3.7	Rekursion bevarad	4
4	Skillnader jämfört med C-implementationen	4

1 Inledning

Denna rapport beskriver en Java-implementation av en parser för ett Pascal-liknande språk. Denna program motsvarar en objektorienterad implementation till den ursprungliga C-implementation från Laboration 1 (`parser.c`, `lexer.c`, `syntab.c`, `optab.c`, `keytoktab.c`). Grammatiken och parsningsstrategin är oförändrade, men programmet är omstrukturerat för att använda klasser, inkapsling och ett antal återkommande designtechniker istället för C:s globala tillstånd och funktions-tabeller.

2 Hur parsern används

Programmet kompileras med `javac *.java` i katalogen som innehåller källfilerna och körs med `java ParserMain`, där källprogrammet skickas in via standard in:

```
java ParserMain < program.pas
```

Parsern läser in hela filen innan analysen börjar, skriver ut källtexten, och analyserar därefter programmet enligt grammatiken: `programhuvud (program namn (input, output) ; )`, en valfri variabeldel (`var lista : typ ;`) och en satsdel (`begin ... end.`) med tilldelningssatser och aritmetiska uttryck (`+`, `*`, parenteser). Under analysen skrivs eventuella syntax- och semantiska fel ut löpande (t.ex. `SYNTAX: Symbol expected ...` och `SEMANTIC: Assign types: ...`), och parsningen fortsätter även efter ett fel istället för att avbrytas, för att kunna rapportera flera fel i samma körning. Efter analysen skrivs symboltabellen ut (`namn`, `roll`, `typ`, `storlek` och `adress` för varje deklarerad identifierare) tillsammans med det totala minnesbehovet, och till sist meddelas `Parse Successful!` eller `Parse Failed!`

Testfallen i katalogen `TestSuite/` (körbara i batch via `testall.sh`) användes för att verifiera att beteendet stämmer mot det förväntade resultatet.

```
for i in TestSuite/*.pas; do
    echo "testing $i"
    java ParserMain < $i
done
```

3 Objektorienterade tekniker och designmönster

C-versionen simulerar objekt genom filstatiska variabler och `static`-funktioner, i Java-versionen görs denna avsikt konkret genom följande tekniker.

3.1 Inkapsling istället för globalt tillstånd

`lookahead`, `currType` och `isOk` är privata instansvariabler i klassen `Parser`, vilket gör varje `Parser`-instans självständig och möjlig att återanvända, till skillnad från C-versionen, där motsvarande tillstånd är globalt för hela processen.

3.2 Konstruktörsinjektion

`Parser` tar emot `Lexer`, `SymbolTable` och `OperatorTable` som konstruktorparametrar istället för att anropa globala funktioner. `ParserMain` skapar och kopplar samman konkreta instanser. Detta gör klasserna oberoende av varandras interna implementation.

3.3 Värdeobjekt

`Token` är en immutable record (`TokenType`, `lexeme`) som returneras som en enhet från `Lexer.nextToken()`, istället för C:s två separata globala accessor-funktioner `get_token()/get_lexeme()`. `OperatorTable` använder på samma sätt en intern record `OpKey` som nyckel i en `HashMap`, vilket ger automatisk `equals/hashCode`.

3.4 Enum med data istället för separata uppslagstabeller

`TokenType` bär sin egen textrepresentation (`lexeme`) och information om det är ett nyckelord (`isKeyword`), vilket ersätter C:s separata tabeller `tokentab/keywordtab` och funktionerna i `keytoktab.c`. `Type` bär på samma sätt sin egen storlek i byte, vilket ersätter `get_type_size()`. Fördelen är en enda källa till sanning.

3.5 Separation av begrepp

C:s enum `toktyp` återanvänds för tre olika saker: lexikal tokenkategori (`id`, `number`), semantisk typ (`integer`, `real`) och roll i symboltabellen (`program`, `var`). I Java är dessa tre separata enum: `TokenType`, `Type` respektive `Role`, vilket gör att kompilatorn förhindrar att de blandas ihop av misstag.

3.6 Inkapslade samlingar istället för parallella arrayer

C:s symboltabell är fem parallella, fastorleks-arrayer (`name[]`, `role[]`, `type[]`, `size[]`, `addr[]`) indexerade för hand. I Java samlas motsvarande fält i en klass `Symbol`, och `SymbolTable` håller dessa i en `List<Symbol>` (bevarar deklara- tionsordning, tillåter dubletter precis som C:s array) samt en `Map<String, Symbol>` för snabb uppslagning (motsvarar C:s linjära sökning `get_ref`). Symbols "setters" är paketprivata, så endast `SymbolTable` får ändra ett symbolobjekt.

3.7 Rekursion bevarad

Sjelva parsningstekniken är en privat metod per grammatikregel som anropar varandra, det vill säga det är oförändrad från C-versionen, eftersom det är en par- sningsteknik snarare än något objektorientering behöver ändra på.

4 Skillnader jämfört med C-implementationen

Utöver de strukturella skillnaderna ovan finns några konkreta skillnader i hur pro- grammet hanterar kantfall. Dessa är medvetna förbättringar snarare än en direkt översättning:

- **Ingen fast buffertstorlek.** C läser källkoden till en array på 1024 tecken (`BUFSIZE`) och skär bort längre filer. Java läser hela strömmen till en `String` utan någon storleksgräns.
- **Tomt indata.** Om filen är tom skriver Java-versionen ut `WARNING: Input file is empty` direkt, istället för att låta hela grammatiken misslyckas token för token (vilket är vad C-versionen skulle göra men inte gjorde vid laboration 1).
- **Kontroll av saknad operand.** `operand()` rapporterar nu `SYNTAX: Operand expected` om varken en identifierare eller ett tal hittas, vilket C-versionens `operand()` saknar.
- **Kontroll av kvarvarande tokens.** Efter att grammatiken är klar kontrol- leras om hela filen faktiskt konsumerats. Om inte, rapporteras `SYNTAX: Extra symbols after end of parse!` följt av de kvarvarande to- kens – något C-versionen inte gör, vilket annars skulle kunna ge en falsk `Parse Successful!` för en fil med skräp efter `end..`
- **Symboltabellens hantering av dubletter.** C:s `addv_name` lägger all- tid till en ny rad, även för en identifierare som just rapporterats som redan

deklarerad. Java-versionen lägger bara till en rad för en identifierare som faktiskt är ny, vilket ger en mer korrekt symboltabell vid felaktig indata.

Sammanfattningsvis behåller Java-implementationen C-versionens grammatik och felåterhämtningsstrategi, men ersätter det globala, tabellbaserade tillståndet med inkapslade objekt, samtidigt som några konkreta felkontroller lagts till som förbättringar utöver originalet.